Mergeable Heaps in Theory and Practice: Binomial, Fibonacci and Pairing Heaps Compared

COSC 520 Assignment 2 — Student number 84298512

AHMED ELGAZWY, University of British Columbia, Canada

We implement three mergeable priority queues from scratch in Python (the binomial, Fibonacci and pairing heaps) behind one interface, analyse the amortized cost of their operations, and benchmark them on 10^7 random keys and on Dijkstra's algorithm over a graph with 10^6 vertices and 10^7 edges. The measured growth rates confirm the analysis: inserts take constant time, the first extract-min after n inserts costs $\Theta(n)$ for the two lazy heaps (8.1 s for the Fibonacci heap at $n = 10^7$, against 0.04 ms for the binomial heap), and a decrease-key to a new minimum costs 1.9–2.4 μs in the lazy heaps against 9.7 μs in the binomial heap. The textbook ranking for Dijkstra's algorithm, however, is reversed. The Fibonacci heap has the best bound, $O(E + V \log V)$, yet it is the slowest (37.5 s, against 26.8 s for the pairing heap): only 0.99 decrease-keys per vertex occur, and its extract-min is 2.4 times slower than the pairing heap's. The pairing heap is the fastest heap in steady state and the smallest, at 102 bytes per item.

Additional Key Words and Phrases: priority queues, mergeable heaps, Fibonacci heap, pairing heap, binomial heap, amortized analysis, Dijkstra's algorithm

1 Introduction

A priority queue stores items with keys and repeatedly hands out the one with the smallest key. Two further operations distinguish the *mergeable heaps* studied here: `decrease_key`, which lowers the key of an item already in the queue, and `merge`, which combines two queues. Dijkstra's shortest-path algorithm [5] calls `decrease_key` whenever it finds a shorter path, and Fibonacci heaps were designed to make that call $O(1)$ amortized, which gives Dijkstra's algorithm its best known bound, $O(E + V \log V)$ [8]. Experimental studies, however, report that the simpler pairing heap [7] is efficient in practice [15] and faster than the Fibonacci heap [13], even though its analysis is still open [6, 14].

This report asks whether the analysis predicts the measured behaviour of the three heaps, implemented by hand in Python, at ten million items. It makes four contributions:

- implementations of the binomial, Fibonacci and pairing heaps behind one handle-based interface, with Dijkstra's algorithm on top and 428 unit tests;
- a complexity analysis of every operation, with the potential functions and bounds it rests on (Section 4);
- a benchmark on a published dataset of 10^7 keys and a 10^7-edge graph (Sections 5–6);
- an explanation of where the measurements follow the analysis and where they do not (Section 7).

Code: https://github.com/ahmedelgazwy/Advanced_DS_assignment_2
Dataset: https://github.com/ahmedelgazwy/Advanced_DS_assignment_2/releases/tag/dataset-v1

2 Why These Structures

I selected these heaps because I wanted to see which one would perform best in a realistic scenario, finding shortest paths with Dijkstra's algorithm, and whether the Fibonacci heap, which has the best theoretical bounds, would also be the fastest in practice. I was also interested in how differently they are built: the binomial heap uses a series of trees, while the pairing heap uses a single tree,

Author's Contact Information: Ahmed Elgazwy, ahmed.elgazwy@ubc.ca, University of British Columbia, Kelowna, British Columbia, Canada.

and I wanted to see whether this leads to a tradeoff between time and space. More generally, the idea of using trees to build a priority queue was interesting to me. In the end there was no tradeoff: the single-tree pairing heap was both the fastest and the smallest of the three.

3 The Three Heaps and Their Applications

All three heaps store `(key, value)` items and implement one interface: `insert` returns a *handle* to the new item, and the caller later passes that handle to `decrease_key` or `delete`. The interface also offers `find_min`, `extract_min` and a destructive `merge`. No library priority queue is used (no `heapq`, no `queue`), and every algorithm is iterative: a pairing heap after n sorted inserts is n levels deep, far beyond Python's recursion limit of 1,000.

Binomial heap [2, 16]. A binomial tree B_k consists of two B_{k-1} trees, one linked below the other's root. A binomial heap is a list of heap-ordered binomial trees with at most one tree of each order, so the orders present are the 1-bits of n: thirteen items form B_3, B_2 and B_0. Merging two heaps works like binary addition, where linking two trees of equal order is the carry [4, Problem 19-2]. We keep a pointer to the minimum root, so `find_min` is $O(1)$. `decrease_key` moves the item up its tree. The textbook version swaps keys between a node and its parent [3, ch. 19], which would leave a caller's handle pointing at the wrong item. We therefore store each item in a separate object that points back to its node, and swap whole items.

Fibonacci heap [8]. A collection of heap-ordered trees whose roots form a circular doubly linked list [4, ch. 19]. The structure is *lazy*: `insert` and `merge` only splice lists, and all tidying is left to `extract_min`, which *consolidates* the roots until no two have the same degree. `decrease_key` cuts a node whose key became smaller than its parent's and moves it to the root list. A node that loses a second child is cut as well (a *cascading cut*); a mark bit records the first loss.

Pairing heap [7]. A single heap-ordered multiway tree. Its only primitive, *link*, makes the larger of two roots the leftmost child of the smaller. `insert`, `merge` and `decrease_key` are one link each (`decrease_key` first cuts the node's subtree out). All restructuring happens in `extract_min`, which combines the root's children by *two-pass pairing*: it links them in pairs from left to right, then links the results one by one from right to left. Each node keeps a pointer to its left sibling (or its parent), so a subtree is cut out in $O(1)$.

Applications. Mergeable heaps with `decrease_key` are the priority queues of Dijkstra's shortest-path algorithm and Prim's minimum-spanning-tree algorithm, the applications for which Fibonacci heaps were introduced [8]. More generally they serve any system that schedules work by priority and must re-prioritize waiting items, such as event-driven simulators and job schedulers, and `merge` combines the queues of two such schedulers in $O(1)$ or $O(\log n)$ time. All three heaps are available in Boost.Heap, with handles for updating keys [1], and the pairing heap is the default of GCC's policy-based `priority_queue` [9].

4 Complexity Analysis

Let n be the number of items, $D(n)$ the largest degree of a node, $t(H)$ the number of trees in the root list, $m(H)$ the number of marked nodes, $\varphi = (1 + \sqrt{5})/2$ and F_k the k-th Fibonacci number. For Dijkstra's algorithm, V and E are the numbers of vertices and edges. An *amortized* bound holds for the total cost of any sequence of operations divided by its length; all three analyses use the potential method [4, §17.3]. Table 1 summarizes the bounds.

Binomial heap. By induction, B_k has 2^k nodes and height k, and its root has degree k [3, Lemma 19.1]. Since the orders present are the 1-bits of n, a heap has at most $\lfloor \log_2 n \rfloor + 1$ trees, and every node has degree at most $\log_2 n$. Union walks the two root lists once, extract-min unions the $\leq \log_2 n$ children of the removed root back in, and decrease-key climbs a path no longer

Table 1. Time and space complexity. Binomial-heap bounds are worst case per operation; the other two are amortized, and a single extract-min of a lazy heap can cost $O(n)$.

Operation	Binomial	Fibonacci (amortized)	Pairing (amortized)
insert	$O(\log n)$; $O(1)$ amortized	$O(1)$	$O(1)$ [11]
find-min	$O(1)$ (min pointer)	$O(1)$	$O(1)$
extract-min, delete	$O(\log n)$	$O(\log n)$	$O(\log n)$ [7]
decrease-key	$O(\log n)$	$O(1)$	$\Omega(\log\log n)$ [6], $O(2^{2\sqrt{\log\log n}})$ [14]
merge	$O(\log n)$	$O(1)$	$O(1)$ [11]
space	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Dijkstra	$O((V+E)\log V)$	$O(E + V\log V)$	$O((V+E)\log V)$

than a tree's height: each is $O(\log n)$. An insert is a binary-counter increment. With the potential $\Phi = t(H)$, an insert that performs c carries costs $1 + c$ and changes Φ by $1 - c$, so its amortized cost is $(1 + c) + (1 - c) = 2 = O(1)$, although a single insert can still cost $\Theta(\log n)$.

Fibonacci heap. With $\Phi(H) = t(H) + 2\,m(H)$ [8], insert raises Φ by 1 and merge leaves it unchanged, so both are $O(1)$. Extract-min costs $O(D(n) + t(H))$ but leaves at most $D(n) + 1$ roots, so the drop in potential pays for the $t(H)$ term and the amortized cost is $O(D(n))$. A decrease-key that triggers c cascading cuts costs $O(c)$ and changes Φ by at most $(t+c)+2(m-c+2)-(t+2m) = 4-c$: amortized $O(1)$. The bound on $D(n)$ gives the structure its name: the i-th child of a node has degree at least $i-2$, so a node of degree k roots at least $F_{k+2} \geq \varphi^k$ nodes, and $D(n) \leq \lfloor \log_\varphi n \rfloor \approx 1.44 \log_2 n$ [4, Lemma 19.4, Cor. 19.5]. Our consolidation table uses the exact bound $\max\{k : F_{k+2} \leq n\}$ in integer arithmetic (33 at $n = 10^7$). Amortization hides spikes: right after n inserts, $t(H) = n$ and the first extract-min does $\Theta(n)$ work.

Pairing heap. Fredman et al. proved $O(\log n)$ amortized time for every operation [7], and Iacono reduced insert and merge to $O(1)$ [11]. Decrease-key is provably *not* $O(1)$: it needs $\Omega(\log\log n)$ amortized time [6, 12], and the best upper bound is $O(2^{2\sqrt{\log\log n}})$ [14]. These bounds come from different analyses and the exact cost is open. In practice the gap is academic: $2^{2\sqrt{\log_2 \log_2 n}}$ only falls below $\log_2 n$ for $n > 2^{16}$, and at $n = 10^7$ the two are 19.2 and 23.3.

Space. All three heaps use $\Theta(n)$ space, one node per item, but their nodes differ: 5 fields for the binomial heap (plus a 3-field item object for handles), 8 for the Fibonacci heap and 5 for the pairing heap. With `__slots__`, CPython 3.10 objects with 3, 5 and 8 fields take 56, 72 and 96 bytes, and a random 31-bit key about 30 bytes, which predicts 158, 126 and 102 bytes per item.

Dijkstra's algorithm performs at most V inserts and extract-mins and at most E decrease-keys, which gives the bounds in Table 1. The E term only matters if many relaxations actually lower a key. If the lengths of the edges entering each vertex are independent and identically distributed, the expected number of decrease-keys is only $O(V \log(1 + E/V))$ [10].

5 Experimental Setup

Machine. Intel Core i7-11800H (8 cores, 24 MB L3 cache), 16 GiB RAM, Windows 11 (build 26200), CPython 3.10.2. The full benchmark ran in 13 minutes.

Data. The dataset has two parts:

- 10^7 keys drawn uniformly from $[0, 2^{31})$ by a seeded generator. The first n keys form the dataset of size n, so one file serves every size.

Table 2. Measured performance at $n = 10^7$ (Dijkstra: $V = 10^6$, $E = 10^7$). In parentheses: the log–log slope fitted for $n \geq 10^4$ (0: constant, 1: linear). Decrease-keys come in batches of $K = 10^5$.

Measurement	Binomial	Fibonacci	Pairing
Insert (μs)	1.48 (0.03)	0.79 (0.06)	0.82 (0.06)
Merge of the two halves (μs)	68.3 (0.24)	22.7 (0.20)	9.2 (0.13)
First extract-min, random keys (ms)	0.04 (0.19)	8,142 (1.06)	1,648 (1.08)
First extract-min, ascending keys (ms)	0.06 (0.18)	8,143 (1.03)	4,027 (1.03)
First extract-min, descending keys (ms)	0.02 (0.06)	7,745 (1.05)	0.01 (0.07)
Extract-min, warm-up batch (μs)	32.1	41.8	87.8
Extract-min, steady state (μs)	33.7 (0.17)	42.5 (0.15)	17.5 (0.17)
Find-min (μs)	0.17	0.11	0.10
Decrease-key, Dijkstra-like (μs)	2.05 (0.11)	1.74 (0.13)	1.75 (0.10)
then one extract-min (ms)	0.05	63.1	86.5
Decrease-key to a new minimum (μs)	9.74 (0.21)	2.37 (0.10)	1.90 (0.11)
then one extract-min (ms)	0.06	138.7	0.01
Memory (bytes per item)	158	126	102
Dijkstra (s)	29.3 (1.15)	37.5 (1.11)	26.8 (1.13)

- A directed graph with $V = 10^6$ vertices and $E = 10^7$ edges with weights uniform in $[1, 1000]$. Its first $V - 1$ edges form a random Hamiltonian path from vertex 0, so every vertex is reachable from the source. The remaining edges join random pairs of distinct vertices.

Both files are published with SHA-256 checksums, and regenerating them reproduces the same bytes. Random keys are the standard average case. The sparse graph ($E = 10V$) is the regime in which the Fibonacci heap's $O(E + V \log V)$ bound should help most.

Workloads. For n on a 1–2–5 ladder from 10^3 to 10^7, each heap is built as two heaps of $n/2$ keys that are then merged. We then time, with $K = \min(10^5, n/10)$:

- the first extract-min;
- a warm-up batch of K extract-mins, kept apart from a steady-state batch of K more;
- K decrease-keys whose new keys are drawn between the current minimum and the old key, as in Dijkstra's algorithm, followed by the one extract-min that pays for them;
- K decrease-keys to a new global minimum, again followed by one extract-min;
- memory, using `sys.getsizeof` over all nodes and keys.

A second experiment builds each heap from n keys in ascending and in descending order and times its first extract-min. A third runs Dijkstra's algorithm on graphs with $V = 10^4, \ldots, 10^6$ vertices and $E = 10V$ edges, and checks that all heaps return identical distances.

Method. Every timed block runs with Python's garbage collector paused, as `timeit` does. Repeated operations are timed in batches of K, because a single $O(1)$ call is too short to resolve. Merge and the first extract-min happen once per build and are timed singly. Each value is the median of three runs for $n \leq 10^6$ (and for Dijkstra $V \leq 10^5$), and of one run above. Growth rates are least-squares slopes of $\log t$ against $\log n$ for $n \geq 10^4$: a slope of 0 means constant time and 1 means linear time.

6 Results

Table 2 lists every measurement at $n = 10^7$, together with its fitted growth rate. The subsections compare each one with the prediction of Section 4.

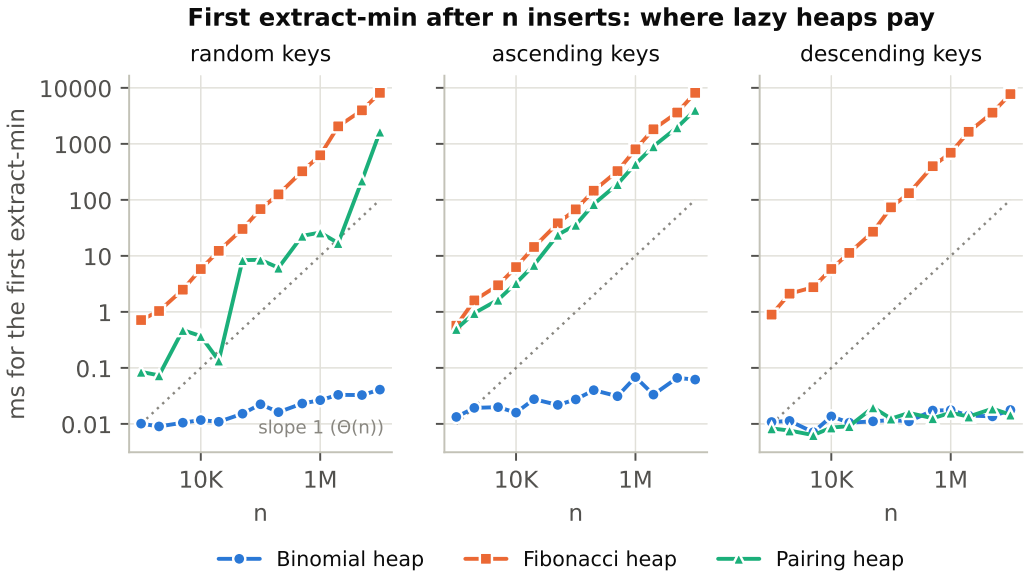


Fig. 1. Time of the first extract-min after n inserts. The dotted line has slope 1. The lazy heaps pay here for all their cheap inserts; the binomial heap was tidy all along.

6.1 Insert and Memory

Inserts take constant time, as predicted: the slopes are 0.03–0.06, and the cost stays between 0.5 and 1.5 μs from 10^3 to 10^7 items. The binomial heap is about twice as slow as the others because an insert may link several trees (the carries), while the lazy heaps only splice one node. Memory is exactly the predicted 158, 126 and 102 bytes per item. The handle-safe design costs the binomial heap 56 bytes per item, its separate item objects.

6.2 The First Extract-Min: Amortization Made Visible

Figure 1 shows the cost that amortization hides. After n inserts the Fibonacci heap has n one-node trees, and its first extract-min consolidates all of them: slope 1.03–1.06 in every key order, and 8.1 s at $n = 10^7$. The pairing heap behaves as its structure predicts. After ascending inserts the root has $n - 1$ children, which the first extract-min must pair (slope 1.03; 4.0 s). After descending inserts every new key is a new minimum, so the heap is a path and the first extract-min costs 0.01 ms. With random keys the root holds only the items inserted after the minimum, so the cost is erratic, but it still grows linearly (slope 1.08). The binomial heap stays below 0.07 ms throughout, as its $O(\log n)$ worst case requires. The pairing heap keeps paying after the first call: its warm-up batch costs 87.8 μs per extract-min, five times its steady state, while the warm-up of the other two heaps matches their steady state. This supports the analysis: the lazy heaps pay for their $O(1)$ inserts later.

6.3 Extract-Min in Steady State

Figure 2 confirms the ranking reported by Larkin et al. [13]: the pairing heap is fastest (17.5 μs at 10^7), ahead of the binomial (33.7 μs) and Fibonacci (42.5 μs) heaps. The growth is only partly logarithmic. From $n = 10^3$ to 10^7, $\log_2 n$ grows by a factor of 2.3, but the cost grows by 3.4–4.0

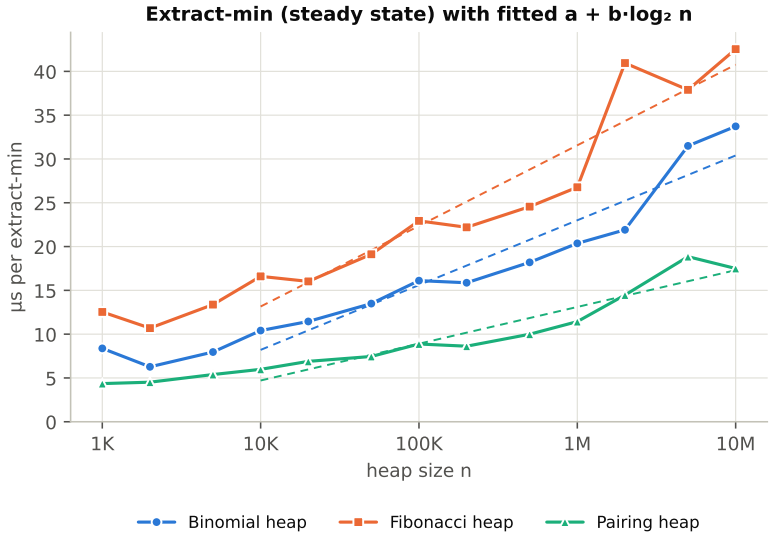


Fig. 2. Steady-state extract-min with fitted $a + b \log_2 n$ curves (dashed, $n \geq 10^4$). The cost grows faster than $\log n$ once the heap outgrows the caches.

times. The $a + b \log_2 n$ fits reach only $R^2 = 0.88$–0.90, with negative intercepts: the cost per level rises with the heap's size. From 10^6 items on, each heap occupies more than 100 MB, far beyond the 24 MB L3 cache, so nearly every node visit can miss the cache. Larkin et al. found the wall-clock time of heaps to be strongly correlated with the L1 cache miss rate [13]. We did not measure cache misses directly, so this is an explanation consistent with the data rather than a proof. The operation count is still $O(\log n)$; what grows is the cost of each pointer it follows.

6.4 Decrease-Key and Its Deferred Cost

When every decreased key becomes the new minimum, the decrease-key path of the binomial heap reaches the root. It then costs 9.74 μs at 10^7 (slope 0.21), against 2.37 and 1.90 μs for the Fibonacci and pairing heaps (slopes 0.10 and 0.11): $O(\log n)$ against $O(1)$, as predicted. With Dijkstra-like keys, which land between the current minimum and the old key, all three heaps cost about 2 μs (Figure 3, left). A binomial bubble-up stops as soon as heap order is restored, which for such keys is usually well before the root, as the gap between 2.05 and 9.74 μs suggests. The right panel shows where the lazy heaps' savings go. After 10^5 Dijkstra-like decrease-keys, the next extract-min takes 63 ms (Fibonacci) and 86 ms (pairing), against 0.05 ms for the binomial heap, because the lazy heaps must re-link the 10^5 subtrees they cut out. The panel grows with n until K reaches its cap of 10^5 at $n = 10^6$, then flattens. When the keys go to new minima, the pairing heap builds a chain and its next extract-min costs only 0.01 ms, while the Fibonacci heap must consolidate 10^5 new roots (139 ms).

6.5 Merge

The binomial merge grows from 11.4 μs at $n = 10^3$ to 68.3 μs at 10^7 (Figure 4), as expected for an operation that walks $O(\log n)$ roots. The Fibonacci and pairing merges perform a fixed number of pointer updates, yet they slow down from 4.4 to 22.7 μs and from 3.5 to 9.2 μs. This contradicts a literal reading of $O(1)$. The work is constant, but a single merge touches roots that were last used

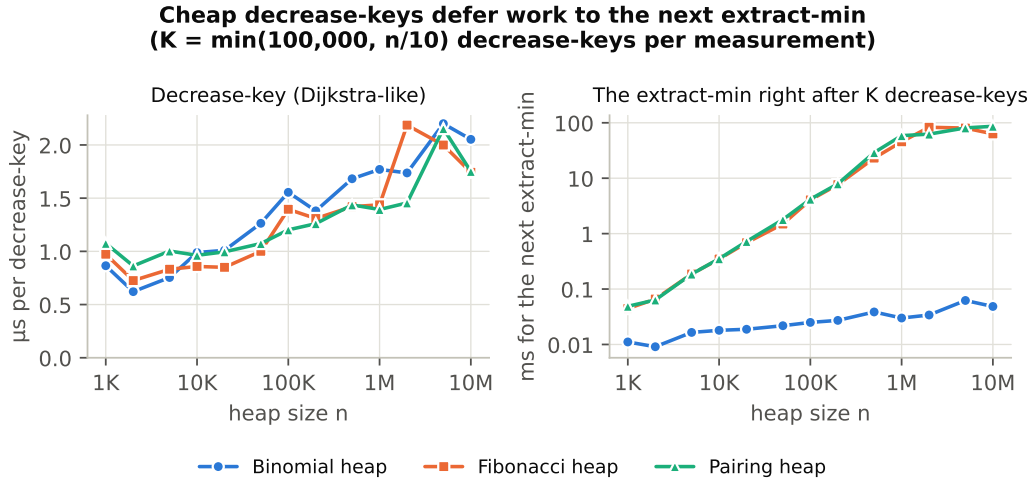


Fig. 3. Left: cost of a Dijkstra-like decrease-key. Right: the single extract-min that follows K of them. The lazy heaps' decrease-keys are cheap because their cost moves into the next extract-min.

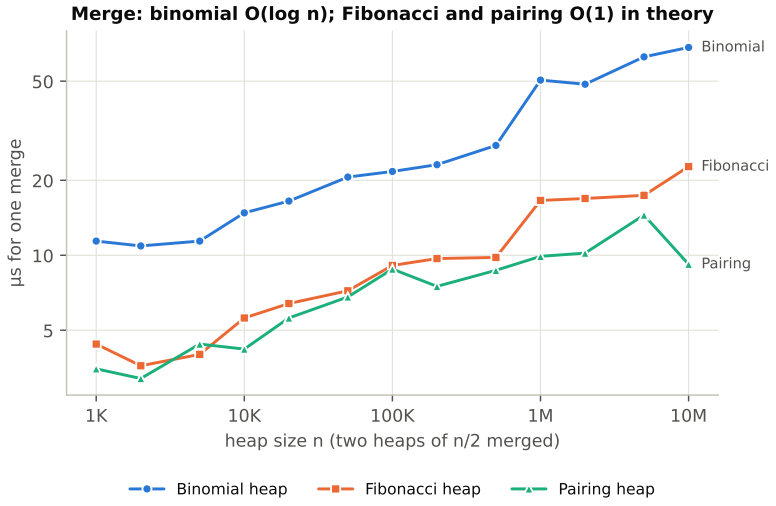


Fig. 4. One merge of two heaps of $n/2$ items. The binomial merge grows with $\log n$. The $O(1)$ merges do constant work but still slow down as the heaps grow.

long before, while the other n nodes were being built. In a large heap those roots are no longer in the CPU cache, so every pointer access waits for main memory. Single-operation timings are also noisy: they are medians of three runs only up to 10^6.

6.6 Dijkstra's Algorithm

All three heaps return identical distances on every graph. At $V = 10^6$ the pairing heap takes 26.8 s, the binomial heap 29.3 s and the Fibonacci heap 37.5 s (Figure 5). This *contradicts* the ranking by

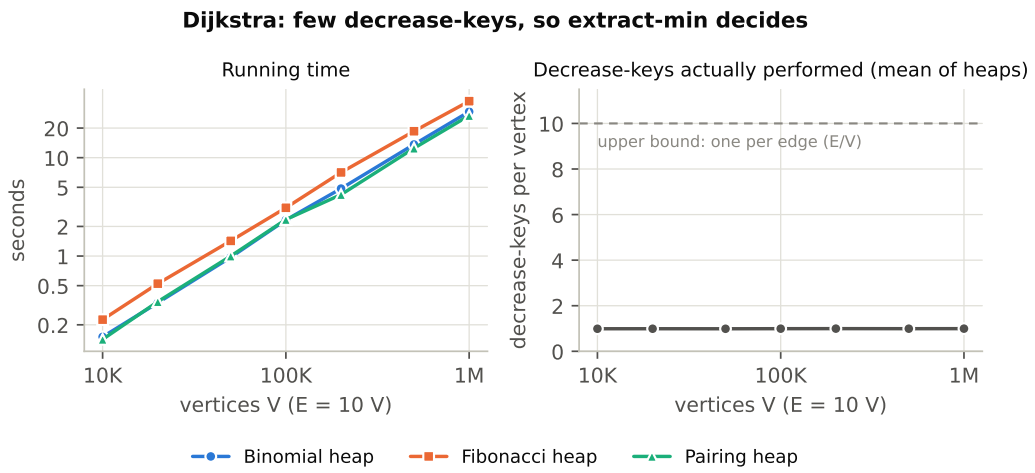


Fig. 5. Dijkstra's algorithm on graphs with $E = 10V$. Left: running time. Right: decrease-keys per vertex, about one, against the upper bound of ten (one per edge).

worst-case bound, in which the Fibonacci heap's $O(E + V \log V)$ beats the others' $O((V + E) \log V)$. The right panel explains the result: only 0.99 decrease-keys per vertex occur (993,926–993,971 at $V = 10^6$), far below the possible $E/V = 10$. This is consistent with the $O(V \log(1+E/V))$ expectation of Goldberg and Tarjan [10]. The Fibonacci heap's $O(1)$ decrease-key therefore has little to save, while every vertex pays for its slower extract-min. The running times grow with slopes 1.11–1.15, slightly above 1, in line with the $\log V$ factor and the cache effects above.

7 Discussion

The analysis predicts the shapes. Every growth rate the analysis predicts appears in the data:

- constant inserts and finds;
- linear first extract-mins for the lazy heaps and logarithmic ones for the binomial heap;
- $O(1)$ against $O(\log n)$ decrease-keys when keys move to the root;
- the deferred cost of laziness, which shows up in the next extract-min.

The two departures, extract-min growing faster than $\log n$ and the $O(1)$ merges slowing down, come from the memory hierarchy, which the RAM model ignores, not from extra operations.

The constants decide the ranking. Asymptotic bounds say nothing about constant factors, and the constants reverse the textbook order for Dijkstra's algorithm. A rough budget shows why. At $V = 10^6$ the run performs 10^6 extract-mins and about 10^6 decrease-keys. At about 2 μs each, all the decrease-keys together cost about 2 s, so even a free decrease-key could save at most 2 s of the 27–38 s total. Extract-min, by contrast, is 2.4 times slower in the Fibonacci heap than in the pairing heap at 10^7 items. Three features make it slower: a larger node (8 fields against 5), consolidation through a degree table over circular lists, and mark bookkeeping. This agrees with Larkin et al., who found that "pairing heaps dominate Fibonacci heaps across the board" [13].

Which heap to use.

- The *pairing heap* is the best default: it is the fastest in steady state and in Dijkstra's algorithm, and the smallest. Its cost is a warm-up after bulk inserts or merges. GCC also makes it the default [9].

- The *binomial heap* never has a slow operation (at most 0.07 ms for any first extract-min, against seconds for the lazy heaps). This suits latency-sensitive uses where one slow operation is worse than a slightly higher average.
- The *Fibonacci heap* is mainly of theoretical value. It pays off only when decrease-keys greatly outnumber extract-mins, which random sparse graphs do not provide.

Threats to validity.

- CPython's interpretive overhead inflates every constant. It probably affects the three heaps similarly, but a compiled implementation could rank them differently.
- All measurements come from one machine.
- Sizes above 10^6 were measured once.
- Pausing the garbage collector removes collection pauses that a real application would experience.

8 Conclusion

We implemented, analysed and benchmarked the binomial, Fibonacci and pairing heaps on up to ten million items. The measurements support the amortized analysis wherever it makes a prediction about growth, including the expensive operations that amortization hides. They also show that the best worst-case bound does not give the fastest program. On Dijkstra's algorithm the Fibonacci heap is the slowest of the three, because decrease-keys are rare and its extract-min is expensive. The pairing heap, whose analysis is still open, is the fastest in practice.

Use of Generative AI

This work was produced with the assistance of Claude Code (Anthropic, Claude Opus 5.5), as allowed by the course policy with disclosure.

- **The assistant** wrote the plan, the three heap implementations, Dijkstra's algorithm, the dataset, benchmark and plotting scripts, the demo and the unit tests, and drafted this report except Section 2.
- **The author**
  - chose the data structures and the application;
  - directed the work in twelve phases, and reviewed and ran each phase's code and tests before it was committed;
  - approved the design decisions recorded in the repository's `docs/review_log.md` (for example, timing Dijkstra-like decrease-keys and separating the warm-up from the steady state);
  - published the dataset;
  - wrote Section 2.

References

[1] Boost C++ Libraries. 2024. Boost.Heap: Data Structures. https://live.boost.org/doc/libs/1_87_0/doc/html/heap/data_structures.html. Version 1.87.0. Accessed 2026-10-24.

[2] Mark R. Brown. 1978. Implementation and Analysis of Binomial Queue Algorithms. *SIAM J. Comput.* 7, 3 (1978), 298–319. doi:10.1137/0207026

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2001. *Introduction to Algorithms* (2nd ed.). MIT Press.

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms* (3rd ed.). MIT Press.

[5] Edsger W. Dijkstra. 1959. A Note on Two Problems in Connexion with Graphs. *Numer. Math.* 1 (1959), 269–271. doi:10.1007/BF01386390

[6] Michael L. Fredman. 1999. On the Efficiency of Pairing Heaps and Related Data Structures. *J. ACM* 46, 4 (1999), 473–501. doi:10.1145/320211.320214

[7] Michael L. Fredman, Robert Sedgewick, Daniel D. Sleator, and Robert E. Tarjan. 1986. The Pairing Heap: A New Form of Self-Adjusting Heap. *Algorithmica* 1, 1–4 (1986), 111–129. doi:10.1007/BF01840439

[8] Michael L. Fredman and Robert Endre Tarjan. 1987. Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms. *J. ACM* 34, 3 (1987), 596–615. doi:10.1145/28869.28874

[9] GNU Project. 2024. The GNU C++ Library: __gnu_pbds::priority_queue. https://gcc.gnu.org/onlinedocs/gcc-14.2.0/libstdc++/api/a01259_source.html. GCC 14.2.0 API reference. Accessed 2026-10-24.

[10] Andrew V. Goldberg and Robert E. Tarjan. 1996. *Expected Performance of Dijkstra's Shortest Path Algorithm.* Technical Report TR-530-96. Department of Computer Science, Princeton University.

[11] John Iacono. 2000. Improved Upper Bounds for Pairing Heaps. In *Algorithm Theory – SWAT 2000 (Lecture Notes in Computer Science, Vol. 1851).* Springer, 32–45. doi:10.1007/3-540-44985-X_5

[12] John Iacono and Özgür Özkan. 2014. Why Some Heaps Support Constant-Amortized-Time Decrease-Key Operations, and Others Do Not. In *Automata, Languages, and Programming – ICALP 2014 (Lecture Notes in Computer Science, Vol. 8572).* Springer, 637–649. doi:10.1007/978-3-662-43948-7_53

[13] Daniel H. Larkin, Siddhartha Sen, and Robert E. Tarjan. 2014. A Back-to-Basics Empirical Study of Priority Queues. In *Proceedings of the Sixteenth Workshop on Algorithm Engineering and Experiments (ALENEX 2014).* SIAM, 61–72. doi:10.1137/1.9781611973198.7

[14] Seth Pettie. 2005. Towards a Final Analysis of Pairing Heaps. In *46th Annual IEEE Symposium on Foundations of Computer Science (FOCS '05).* IEEE, 174–183. doi:10.1109/SFCS.2005.75

[15] John T. Stasko and Jeffrey Scott Vitter. 1987. Pairing Heaps: Experiments and Analysis. *Commun. ACM* 30, 3 (1987), 234–249. doi:10.1145/214748.214759

[16] Jean Vuillemin. 1978. A Data Structure for Manipulating Priority Queues. *Commun. ACM* 21, 4 (1978), 309–315. doi:10.1145/359460.359478